

Week 5 Technical Report - Dynamic Programming

Project repository (complete source, tests, benchmarks and results):

<https://github.com/RDeibel2025/CTX-Advanced-Algorithms>

Dynamic programming: `src/dp/` · demonstration: `examples/week5_demo.py`

Benchmark: `benchmarks/week5_dp_benchmark.py`

Robert Deibel · CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026



Complete project repository

<https://github.com/RDeibel2025/CTX-Advanced-Algorithms>

1. Executive Summary

Memoization and tabulation compute the same subproblems and differ only in who decides the order, and every trade-off measured here followed from that. Tabulation won wherever the whole table was needed, by 3.0x and a tenth of the memory on 1,000-character LCS, while top-down won on knapsack as long as the subproblem space stayed sparse, beating it 4.0x at 10 items and losing from 25 on. The lazy order also spends the interpreter's stack: memoized LCS stops working at 600 characters where tabulation is untroubled.

2. Methodology

Environment. Apple M2 Max, 12 cores, 64 GB RAM; macOS 14.5 (arm64); CPython 3.12.4. Timing runs through `src/utils/timer.py` on the Week 1 discipline: collection paused per run, warm-ups discarded, mean, standard deviation, minimum and maximum reported. Peak memory is tracemalloc in a separate pass, since tracing costs about an order of magnitude in speed. Call counts and depths come from a counter object rather than a global, on code paths separate from the timed ones.

Inputs. Fibonacci at $n = 10$ to 45. Knapsack on seeded instances (weights 1-50, values 1-100) swept two ways: item count 10 to 200 at capacity 1,000, and capacity 100 to 1,000 at 50 items. LCS on random four-letter strings of 10 to 1,000 characters. Seed 42 throughout.

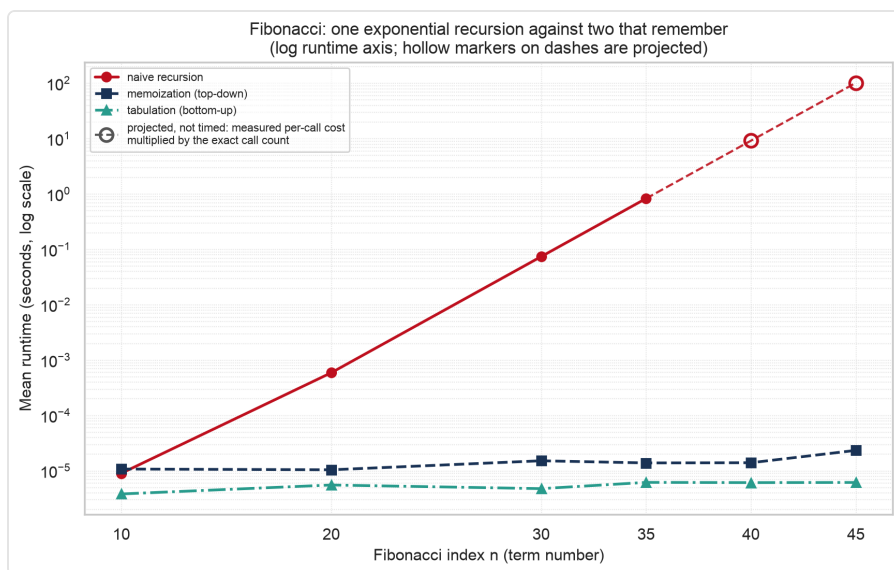
Repetitions. Each cell runs a pilot and takes its policy from that cost: 5 runs after 2 warm-ups below 50 ms, 3 after 1 below 500 ms, 2 otherwise. One policy for all would waste an hour on naive Fibonacci or under-sample the rest. Every CSV row records the count used.

Reductions and projections, recorded rather than silent. Naive Fibonacci is measured to $n = 35$ and projected at 40 and 45 from the measured per-call cost, 27.58 ns over 29,860,703 calls, times the exact count $2 \cdot F(n+1) - 1$, which the instrumented counter reproduces at every measured size; $n = 45$ would take about 101 seconds a run. The recursive knapsack is capped at 20 items and the recursive LCS at 14 characters, while the DP variants cover the full ranges. Every row carries a `measurement` column, and `speedup_vs_recursive` stays

blank wherever the baseline was projected or never run, because a ratio of a measurement to an estimate is not a measurement. It also probes where memoized LCS fails at the default limit of 1,000, with tabulation on identical inputs as the control, before raising it to 30,000. Every figure below is recomputed from the CSVs by [tools/week5_facts.py](#).

3. Results

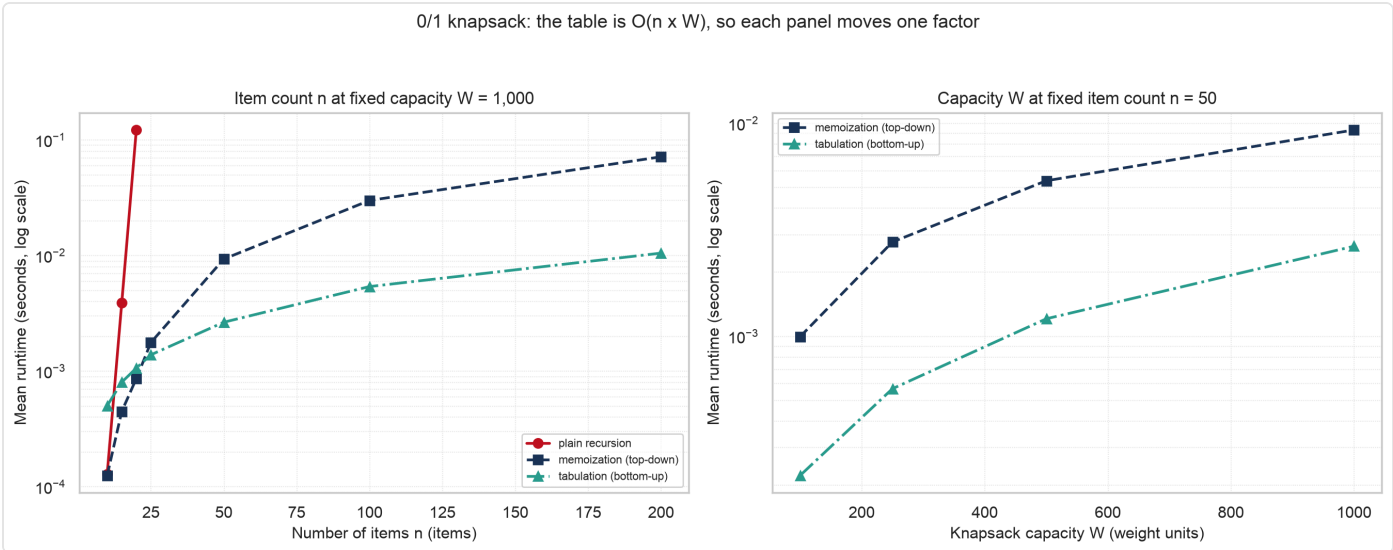
3.1 Fibonacci



Naive recursion goes from 0.0090 ms at $n = 10$ to 823.4 ms at $n = 35$, a factor of 91,653 for a 3.5x rise in n , tracking its call count (177 to 29,860,703). Both DP forms are flat across the same range: memoization 0.0104 to 0.0233 ms, tabulation 0.0038 to 0.0062 ms. At $n = 35$ that is 59,628x and 133,888x. The projected $n = 45$ point puts naive recursion at 101 seconds against memoization's 0.0233 ms.

Two smaller results matter. At $n = 10$ memoization is no faster than plain recursion, 0.0107 ms against 0.0090: 177 calls is too few for a dict to repay its own overhead, so memoization has a crossover of its own. Tabulation beats memoization by 1.9x to 3.8x at every size, making fewer calls ($n+1$ against $2n-1$) and holding 0.1 KiB against 4.1 KiB, because a rolling pair allocates no table.

3.2 Knapsack

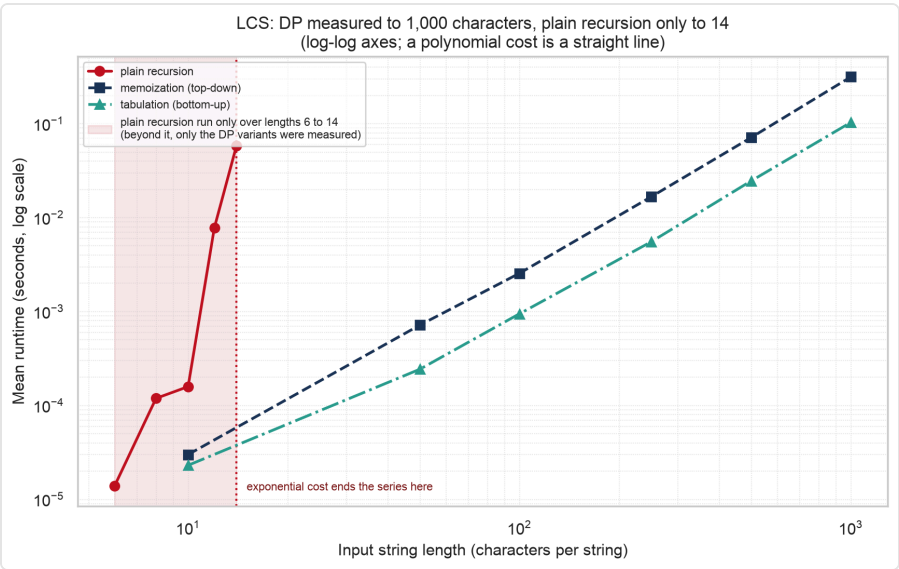


Plain recursion makes $2^{(n+1)} - 1$ calls and behaves like it: 0.129 ms at 10 items and 121.680 ms at 20, where memoization is 141.5x faster.

The result worth the space is between the two DP forms. Top-down was faster at 10 items (0.125 ms against 0.501), 15 and 20; bottom-up took the lead at 25 (1.383 against 1.765) and kept it, reaching 6.8x at 200 (10.473 against 71.287). The cell counts say why: the memo stored 504 of 10,010 cells at $n = 10$, 5.0% of the table, rising through 16.2% at 20, 25.8% at 25 and 89.2% at 200. Top-down wins while the subproblem space is sparse and loses once it is not, and the crossover sits where occupancy passes roughly a quarter.

Capacity behaves as $O(n \cdot W)$ predicts: at 50 items a 10x rise in W took tabulation from 0.221 to 2.644 ms, a factor of 11.9.

3.3 Longest common subsequence



Recursion is $O(2^{(m+n)})$ and reaches 58.2 ms at 14 characters, which is where the series ends. Both DP forms are $O(m*n)$ and plot straight on log-log axes: tabulation 0.023 ms at 10 characters to 102.7 ms at 1,000, memoization 0.030 to 313.3 ms.

Here the measured curve departs from theory. Over that 100x range $m*n$ predicts 10,000x; tabulation measured 4,477x. The shortfall is fixed overhead at the small end, where a 10-character call is mostly interpreter work, inflating the baseline and understating the growth. Between 100 and 1,000 characters, where that overhead is negligible, the factor is 109.1x against 100x predicted. Tabulation is 3.0x faster at 1,000 characters and holds 11.4 MB against 117 MB.

4. Discussion

Both DP forms need the same two properties, optimal substructure and overlapping subproblems (CLRS §14.3), and compute the same subproblem values. What differs is who fixes the order: top-down asks for a subproblem when it needs one and discovers the order lazily at run time, while bottom-up fixes it in advance and fills everything, needed or not. CLRS draws that distinction over rod cutting (§14.1), and Bellman framed the method itself as a choice of ordering over a multistage decision (1966).

Three results follow directly. **Constant factors:** where the whole table is needed anyway, the eager form has no call frames and no dict hashing, the 2-to-4x on Fibonacci and the 3.0x on LCS. **Sparsity:** where it is not needed, laziness is the right answer, the 5.0% occupancy and the 4.0x win at 10 knapsack items. **Space:** knowing the order in advance is what lets the table collapse, so `knapsack_tab_rolling` keeps one row of $W+1$ cells and Fibonacci's rolling pair the same trick, 0.1 KiB at $n = 45$. Memoization cannot do that, because it does not know what it will still be asked for.

And it breaks. Laziness borrows the interpreter's call stack, so memoized depth grows with the input: 1,696 frames at 1,000 characters against tabulation's 1 at every size. At the default limit of 1,000, memoized LCS succeeded to 500 characters and raised `RecursionError` at 600, 800 and 1,000, while tabulation returned the right answer at every one. Choosing who decides the order is also choosing which stack you spend, a property of the runtime rather than of the recurrence.

5. Case Study: sequence alignment

LCS is the same machinery as biological sequence alignment. Needleman and Wunsch (1970) fill a table over prefix pairs with a recurrence differing from §3.3's only in scoring, since matches, mismatches and gaps carry weights rather than a one-or-nothing match, and recover the alignment by walking the table backwards as `lcs_reconstruct` does. Scale turns the stack question practical: two 1,000-character sequences already needed 1,696 frames and 117 MB of table, and real alignments run to millions of bases. That is why production aligners are written bottom-up over a rolling row.

6. Conclusion

The first decision is not between the two DP forms: plain recursion is a different asymptotic class, and either form erases it, by 141.5x on knapsack at 20 items and 59,628x on Fibonacci at 35. Choosing between them afterwards is choosing which resource to spend. Tabulation pays for cells it may never need and buys a bounded stack and a collapsible table; memoization pays for call frames and hashing and buys the right to touch only the subproblems the input reaches. That bought a 4.0x win at 10 knapsack items, cost 6.8x at 200, and on a pair of 1,000-character strings cost the run outright.

References

- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed., Ch. 14: §14.1 Rod Cutting, pp. 360-369; §14.3 Elements of Dynamic Programming, pp. 378-386; §14.4 Longest Common Subsequence, pp. 386-392
- Bellman, R. (1966). "Dynamic Programming." *Science*, 153(3731), 34-37
- Needleman, S. B., and Wunsch, C. D. (1970). "A general method applicable to the search for similarities in the amino acid sequence of two proteins." *Journal of Molecular Biology*, 48(3), 443-453
- Amakobe, M. (2025). *Advanced Computational Algorithms*, 2nd ed., Ch. 6, §6.1-6.4

AI use

Claude (Claude Code) drafted the code, tests and this report's first draft from a specification I wrote from the instructions and required reading. Every measurement comes from running the benchmark on my machine, and every figure is recomputed from the CSVs rather than transcribed. Full disclosure: [docs/AI_USE.md](#) .